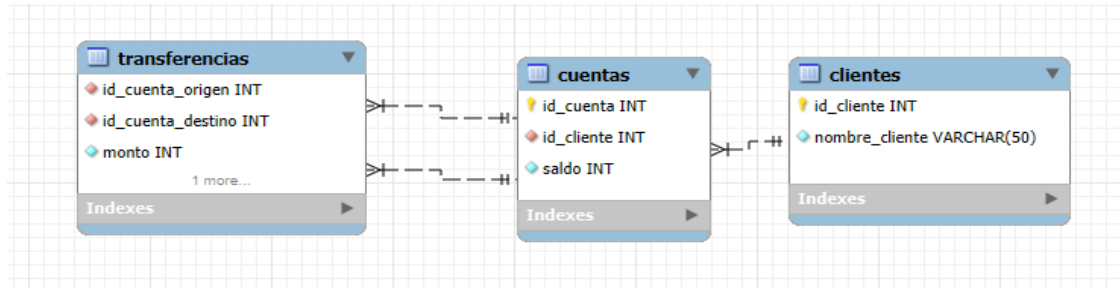


En este laboratorio, veremos la creación de procedimientos almacenados y manejo de transacciones en MySQL. Para ello, en primer lugar, trabajaremos con 3 tablas que simulan las cuentas bancarias de clientes y el registro de transferencias de dinero entre sus cuentas, creadas en el schema INFO209, en el Práctico Semana 4.



1. Manejo de Transacciones.

La semana anterior, compilamos el procedimiento almacenado **p_transferir(cta_origen,cta_destino,monto)**, que transfiere el monto de dinero entre ambas cuentas y registra la transferencia en la tabla respectiva.

Aun cuando dicho procedimiento incorpora manejo básico de errores, actualmente permite que una cuenta con saldo insuficiente transfiera dinero a otra.

- Realice una transferencia, utilizando **p_transferir**, por un monto superior al saldo de la cuenta de origen, y vea qué sucede. ¿Arroja algún error? ¿Qué saldo queda en las cuentas de origen y destino?
- Realice una transferencia hacia una cuenta inexistente y verifique el resultado de esta operación en las tablas involucradas.
- Para evitar sobregiros, podemos agregar una restricción de integridad que evite que tengamos saldo negativo. Esto puede hacerse de varias maneras:
 - Borrando la tabla de cuentas, modificando el script y cargando de nuevo los datos.
 - Alterando la definición de la tabla, sin borrar la tabla. Para ello, debemos corregir previamente los datos que no cumplen con la restricción. Ejm:
update cuentas **set** saldo=0 **where** saldo <0;
alter table cuentas **add constraint** ck_saldo **check**(saldo>=0);
 - También podemos definir el tipo de dato de saldo como **"int unsigned"**.

Pruebe ahora qué sucede al realizar una transacción que provocaría sobregiro.

Revise los datos en las tablas.

- d) Si ahora queremos dejar registro de las transacciones no realizadas, Modifique la tabla de transferencias, para agregar un estado de transacción. Se espera siempre un registro de la transferencia, pero que, si falla la transacción, quede indicado esto en la tabla de transferencias. Si la transacción ocurre correctamente, debiera quedar registrado ese hecho.

Hint: Podemos definir un atributo a la tabla de tipo “enum”, como por ejemplo:

estado enum ('OK', 'No OK') default 'OK'

Ahora debe **modificar el procedimiento almacenado**, para que, si la transacción es correcta, se registren los movimientos en las cuentas, y también la transferencia, en estado “OK”. Si la transacción no es correcta, no deben registrarse movimientos en las cuentas, sino sólo el registro de transferencia, en estado “No OK”.

2. Verificación de propiedades ACID en MySQL.

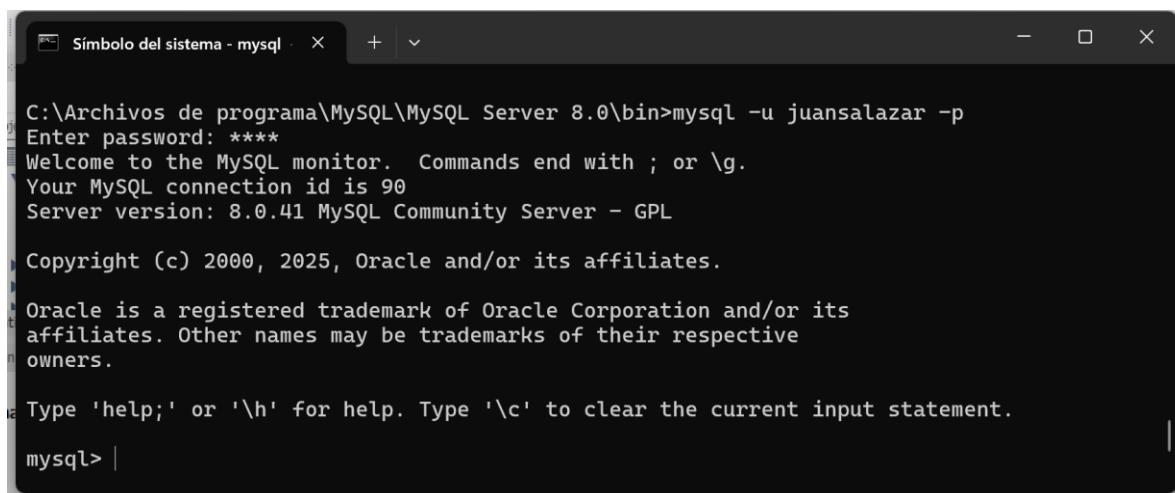
Para ello, trabajaremos en **MySQL Workbench** y **SQL vía Símbolo del Sistema**, simulando lo que podría ocurrir con 2 usuarios distintos que interactúan con nuestra base de datos.

(MySQL Workbench maneja una única conexión a la base de datos en las diferentes pestañas, entonces necesitamos una segunda conexión para ilustrar la concurrencia)

En Símbolo del Sistema, deben llegar a la carpeta que contenga el archivo **mysql.exe** con el comando **cd**, el cual debería estar en **C:\Archivos de programa\MySQL\MySQL Server 8.0\bin** si no cambiaron la ruta de instalación.

Una vez lleguen a la carpeta, genere una **conexión con su usuario**. Use el siguiente comando: **mysql -u usuario -p**

Y finalmente ingresen su contraseña de MySQL para poder usar los comandos SQL.



```
Símbolo del sistema - mysql
C:\Archivos de programa\MySQL\MySQL Server 8.0\bin>mysql -u juansalazar -p
Enter password: ****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 90
Server version: 8.0.41 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> |
```

Para esta parte del práctico, vamos a deshabilitar el autocommit en cada una de nuestras conexiones, para que podamos ver cómo opera el manejo de transacciones, paso a paso.



Para el Símbolo del Sistema se **deshabilita** con el comando: **SET autocommit = 0;**

Y para **habilitar** después el autocommit, use el comando: **SET autocommit = 1;**

A continuación, haremos pruebas **similares a las que vimos en la PPT**, siguiendo las instrucciones de nuestro procedimiento almacenado base:

- Rebaja de saldo en cuenta de origen
- Incremento de saldo en cuenta de destino
- Registro de transferencia

Adicionalmente, probaremos cómo se reflejan los cambios al hacer commit o rollback. Las pruebas que haremos implican realizar transacciones en dos conexiones.

Recordatorio: Puedes cancelar una acción en SQL de Símbolo del Sistema con **ctrl + C**, y salir del modo SQL de Símbolo del Sistema simplemente escribiendo **exit**.

- Ejecutamos 2 transacciones concurrentes, que afectan **registros de la tabla de cuentas** con `id_cuenta` distintos (ejemplo: un movimiento desde la cuenta 1 a la 2, y otro movimiento entre la cuenta 3 y la 4).
 - ¿Se producen bloqueos?
 - ¿Qué versión de los datos vemos en cada cuenta, después de ejecutar las instrucciones, pero antes de hacer commit? Haga **SELECT** a la tabla de cuentas.
 - ¿Cambian los datos que ven las otras cuentas si hacemos rollback?
- Ejecutamos 2 transacciones concurrentes, que afectan a las mismas cuentas. Por ejemplo, dos movimientos de dinero de la cuenta 1 a la 2.
 - ¿Se producen bloqueos?
 - ¿Qué versión de los datos vemos en cada cuenta, después de ejecutar las instrucciones, pero antes de hacer commit? Haga **SELECT** a la tabla de cuentas.
 - ¿Cambian los datos que ve la otra conexión, si hacemos rollback en una de las conexiones?
- Ejecutamos 2 transacciones concurrentes, y una de ellas podría generar sobregiro (ejm: 2 transferencias simultáneas por el saldo total de la cuenta).
- Ejecutamos una transacción con alguna falla (ejm: cuenta origen inexistente). Realizamos las 3 instrucciones, y al final hacemos commit. ¿Queda inconsistente la información?

- e) La función **sleep(x)** en MySQL produce un retardo de x segundos. Modifique el procedimiento almacenado de manera que se genere un retardo de 10 segundos entre cada instrucción insert o update. Busque en google cómo usar la función sleep(10) dentro de un procedimiento almacenado.
- Ejecute el procedimiento de manera simultánea, realizando en una conexión una transferencia desde la cuenta 1 a la 2, y desde la otra conexión una transferencia entre la cuenta 2 y la 1.
 - Vea qué sucede. ¿Cómo lo explica?

3. Triggers

Un **trigger** en MySQL es una acción que se ejecuta cuando insertamos, actualizamos o eliminamos registros en una tabla. Es decir, es un conjunto de **instrucciones** que se ejecuta, **gatillada por un evento**.

Se le pide implementar una versión simplificada del procedimiento almacenado p_transferir, que sólo inserte la transacción en la tabla de transferencias, debiendo dicha inserción gatillar acciones en la tabla de cuentas (actualización de saldos). En ese caso, el procedimiento almacenado podría ser tan simple como lo siguiente:

```
DELIMITER $$
DROP PROCEDURE IF EXISTS p_transferir$$
CREATE PROCEDURE p_transferir(
    IN cta_origen int, IN cta_destino int, IN monto int,
    OUT mensaje varchar(255)
)
BEGIN
    declare exit handler for SQLEXCEPTION
    begin
        set mensaje = 'transacción no realizada';
        rollback;
    end;

    start transaction;

    insert into transferencias values (cta_origen, cta_destino, monto, now());

    commit;
end
$$
DELIMITER ;
```

- a) Por simplicidad, vuelva la tabla de transferencias a su versión original (sin atributo de Estado).
- b) Investigue cómo crear un trigger en MySQL que, ante una inserción en la tabla de transferencias:
- Incremente el saldo en la cuenta de origen.
 - Rebaje el saldo en la cuenta de destino.

La sintaxis de un trigger en MySQL es algo como esto:

```
CREATE TRIGGER trg_transferencia_before_insert
BEFORE INSERT ON transferencias
FOR EACH ROW
BEGIN
    ...
END;
```

- c) Pruebe la nueva versión del procedimiento almacenado p_transferir.
- “Happy path”, una transacción que debiera ser correcta (cuentas existen, saldos correctos).
 - Casos con errores.